LAB 7

Fuzz Testing for Input Validation Bugs

Tennis Scoring Application

For Lab 7, I used Hypothesis to fuzz the three tennis scoring functions from my Lab 6 application. The repository's Lab 6 code is named `tennis_scoring.py`; I used the same function logic in the `processor.py` file requested by this handout.

What the fuzz tests found

The starting version did not validate every input consistently. For example, `set_winner(-1, 0)` returned `None` instead of rejecting a negative set score. Boolean values were treated like integers, invalid types could produce unrelated Python errors, and `match_status` accepted a zero or even `best_of` value.

Function	Issue found	Correction
game_score	Boolean, non-integer, and negative points were not handled consistently.	Validate both points before calculating the score.
set_winner	Negative or invalid set scores could be accepted or fail indirectly.	Validate both set scores before deciding a winner.
match_status	Set scores and best_of were not validated.	Validate set scores and require a positive odd best_of.

How I fixed it

I added a shared input check that rejects booleans and other non-integer values with `TypeError`, and rejects negative values with `ValueError`. For match length, I also check that `best_of` is a positive odd integer. This keeps the three functions' input rules clear and predictable.

Testing and result

The test file uses Hypothesis strategies to generate normal scores and unexpected values for all three functions. The final run completed with 10 pytest tests passing. The screenshot section includes the original run, a minimized failing fuzz example, the corrected run, and the passing test run.

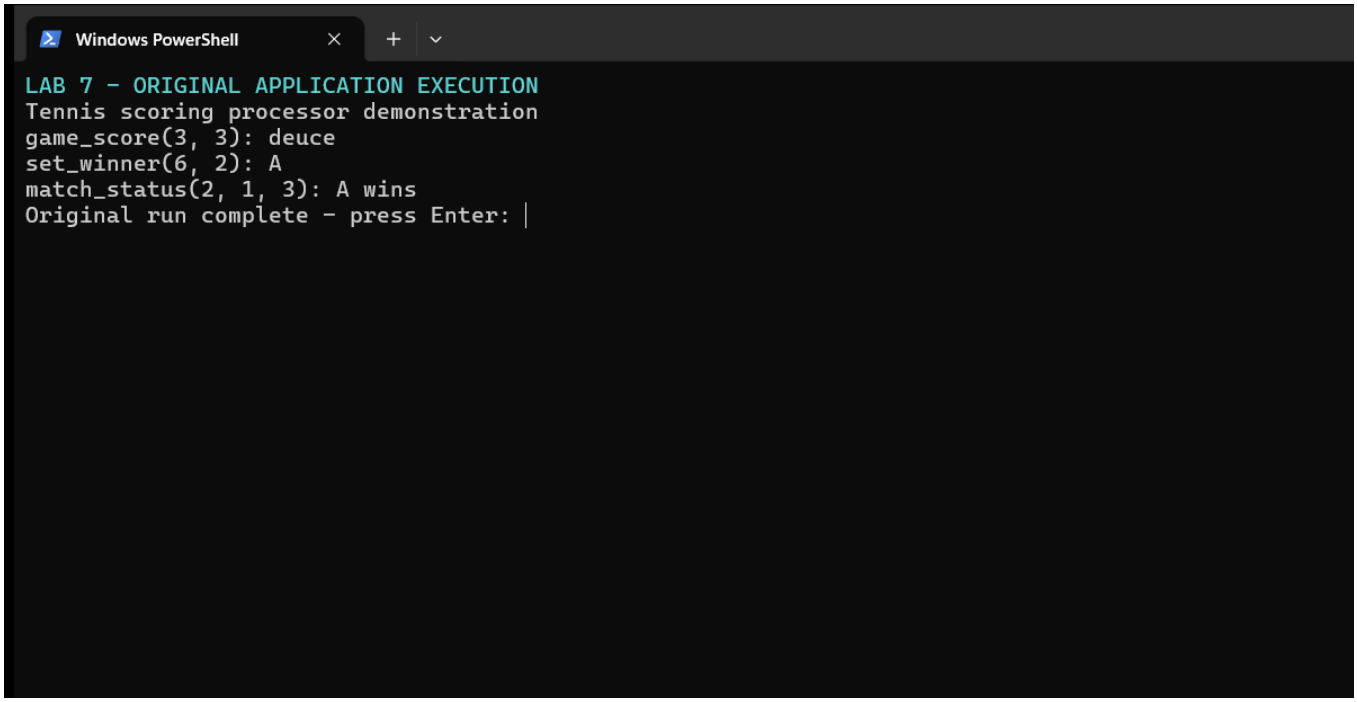
Commands used

```
py -m pip install -r requirements.txt
py processor.py
py -m pytest -q test_processor.py
```

The full `processor.py` and `test_processor.py` source files are included at the end of this report. The matching files and screenshots are also in the Lab 7 folder in my GitHub repository.

1. Original application execution

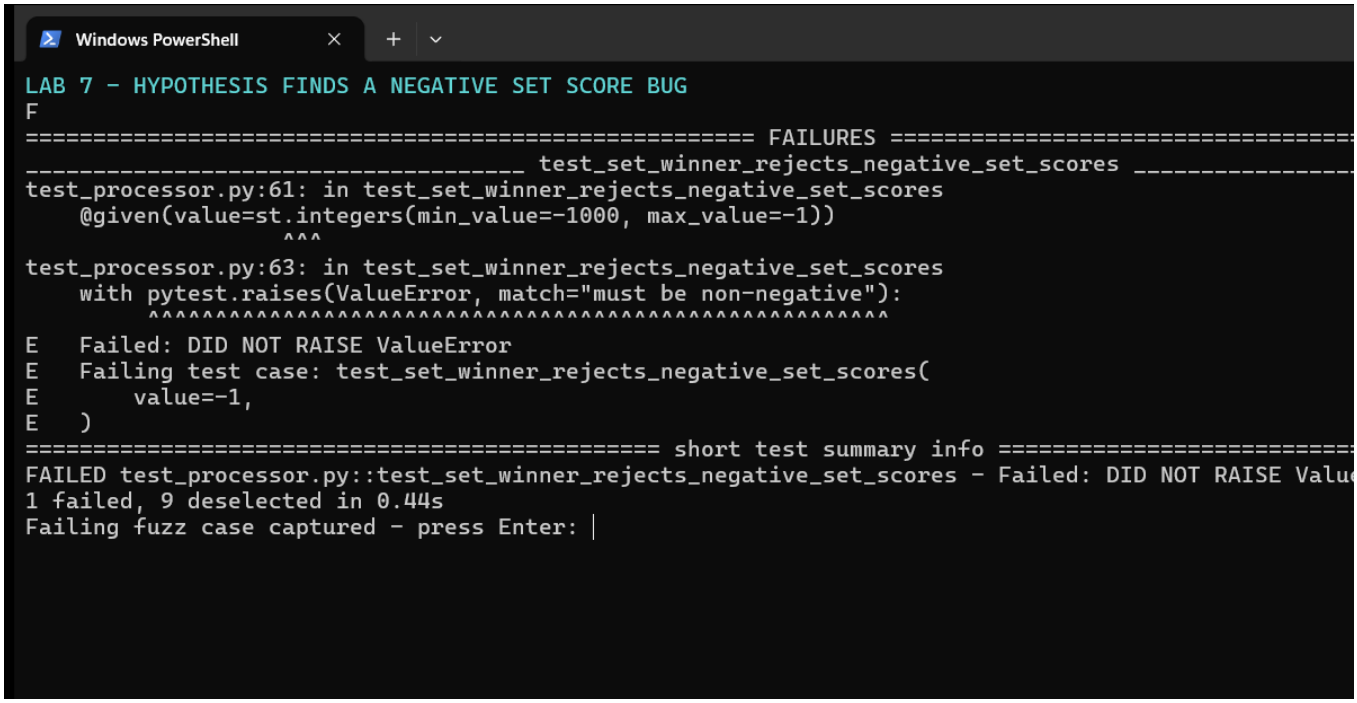
This is the starting version before input validation fixes.



Captured from the PowerShell terminal during the actual application or test run.

2. Fuzzing failure

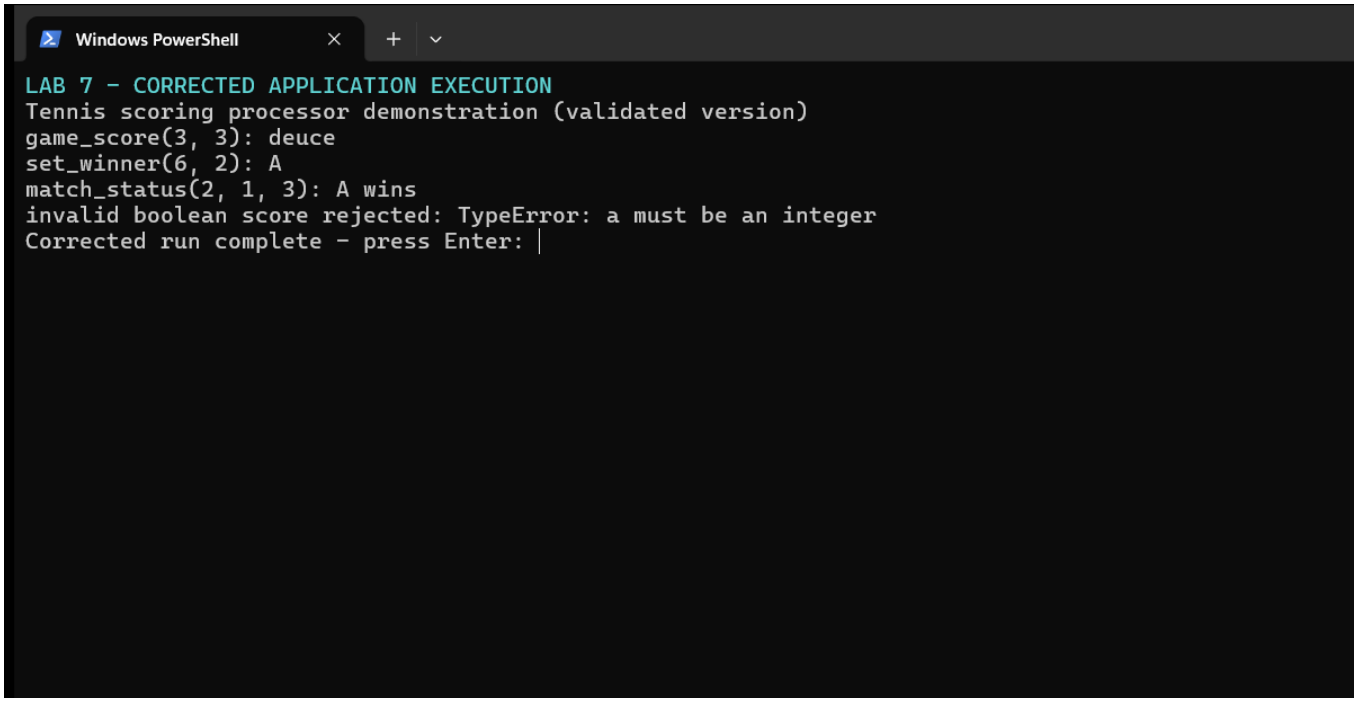
Hypothesis minimized a failing case to a negative set score of -1; the starting function returned without raising the expected validation error.



Captured from the PowerShell terminal during the actual application or test run.

3. Corrected application execution

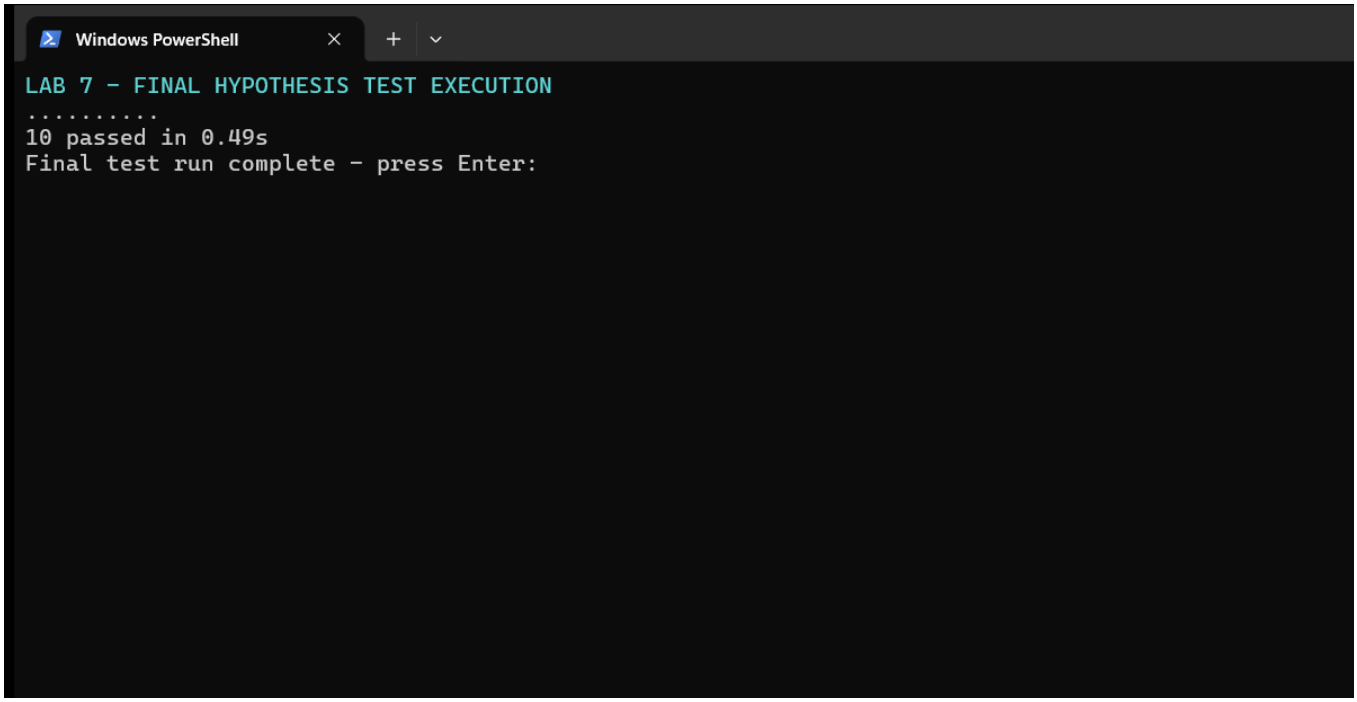
The corrected program demonstrates normal scoring and rejects a boolean score with a clear TypeError.



Captured from the PowerShell terminal during the actual application or test run.

4. Final fuzz test execution

All 10 Hypothesis test functions pass against the corrected application.



Captured from the PowerShell terminal during the actual application or test run.

Appendix A - Fixed processor.py

Source file: `processor.py`

```
"""Tennis scoring processor with input validation (Lab 7 fixed version)."""

POINTS = ["0", "15", "30", "40"]

def _require_nonnegative_integer(value, name):
    if isinstance(value, bool) or not isinstance(value, int):
        raise TypeError(f"{name} must be an integer")
    if value < 0:
        raise ValueError(f"{name} must be non-negative")

def game_score(a, b):
    _require_nonnegative_integer(a, "a")
    _require_nonnegative_integer(b, "b")
    if a >= 3 and b >= 3:
        if a == b:
            return "deuce"
        if a - b >= 2:
            return "game A"
        if b - a >= 2:
            return "game B"
        if a > b:
            return "advantage A"
        return "advantage B"
    if a >= 4:
        return "game A"
    if b >= 4:
        return "game B"
    return POINTS[a] + "-" + POINTS[b]

def set_winner(a, b):
    _require_nonnegative_integer(a, "a")
    _require_nonnegative_integer(b, "b")
    if a >= 6 and a - b >= 2:
        return "A"
    if b >= 6 and b - a >= 2:
        return "B"
    if a == 7 and b == 6:
        return "A"
    if b == 7 and a == 6:
        return "B"
    return None

def match_status(sets_a, sets_b, best_of):
    _require_nonnegative_integer(sets_a, "sets_a")
    _require_nonnegative_integer(sets_b, "sets_b")
    if (
        isinstance(best_of, bool)
        or not isinstance(best_of, int)
        or best_of <= 0
        or best_of % 2 == 0
    ):
        raise ValueError("best_of must be a positive odd integer")
    needed = best_of // 2 + 1
    if sets_a == needed:
        return "A wins"
    if sets_b == needed:
        return "B wins"
    if sets_a == needed - 1 and sets_b == needed - 1:
        return "decider"
    return "in progress"

def main():
    print("Tennis scoring processor demonstration (validated version)")
    print(f"game_score(3, 3): {game_score(3, 3)}")
    print(f"set_winner(6, 2): {set_winner(6, 2)}")
    print(f"match_status(2, 1, 3): {match_status(2, 1, 3)}")
    try:
        game_score(True, 0)
    except (TypeError, ValueError) as error:
        print(f"invalid boolean score rejected: {type(error).__name__}: {error}")

if __name__ == "__main__":
    main()
```

Appendix B - Hypothesis test_processor.py

Source file: `test_processor.py`

```
"""Hypothesis-based fuzz tests for the tennis scoring processor."""

import pytest
from hypothesis import given, settings, strategies as st

from processor import game_score, match_status, set_winner

non_integer = st.one_of(
    st.none(),
    st.booleans(),
    st.floats(allow_nan=True, allow_infinity=True),
    st.text(),
    st.lists(st.integers()),
    st.dictionaries(st.text(), st.integers()),
)

fast_fuzz = settings(max_examples=60, deadline=None)

@fast_fuzz
@given(a=st.integers(min_value=0, max_value=30), b=st.integers(min_value=0, max_value=30))
def test_game_score_returns_a_valid_score_for_nonnegative_points(a, b):
    result = game_score(a, b)
    valid_results = {
        "0-0", "0-15", "0-30", "0-40", "15-0", "15-15", "15-30", "15-40",
        "30-0", "30-15", "30-30", "30-40", "40-0", "40-15", "40-30", "40-40",
        "deuce", "advantage A", "advantage B", "game A", "game B",
    }
    assert result in valid_results

@fast_fuzz
@given(value=non_integer)
def test_game_score_rejects_non_integer_points(value):
    with pytest.raises(TypeError, match="must be an integer"):
        game_score(value, 0)

@fast_fuzz
@given(value=st.integers(min_value=-1000, max_value=-1))
def test_game_score_rejects_negative_points(value):
    with pytest.raises(ValueError, match="must be non-negative"):
        game_score(value, 0)

@fast_fuzz
@given(a=st.integers(min_value=0, max_value=30), b=st.integers(min_value=0, max_value=30))
def test_set_winner_returns_a_valid_result_for_nonnegative_sets(a, b):
    assert set_winner(a, b) in {"A", "B", None}

@fast_fuzz
@given(value=non_integer)
def test_set_winner_rejects_non_integer_set_scores(value):
    with pytest.raises(TypeError, match="must be an integer"):
        set_winner(value, 0)

@fast_fuzz
@given(value=st.integers(min_value=-1000, max_value=-1))
def test_set_winner_rejects_negative_set_scores(value):
    with pytest.raises(ValueError, match="must be non-negative"):
        set_winner(value, 0)

@fast_fuzz
@given(
    sets_a=st.integers(min_value=0, max_value=10),
    sets_b=st.integers(min_value=0, max_value=10),
    best_of=st.sampled_from([1, 3, 5, 7, 9]),
)
def test_match_status_returns_a_valid_result_for_valid_inputs(sets_a, sets_b, best_of):
    assert match_status(sets_a, sets_b, best_of) in {
        "A wins", "B wins", "decider", "in progress"
    }

@fast_fuzz
@given(value=non_integer)
def test_match_status_rejects_non_integer_set_scores(value):
    with pytest.raises(TypeError, match="must be an integer"):
        match_status(value, 0, 3)

@fast_fuzz
@given(value=st.integers(min_value=-1000, max_value=-1))
def test_match_status_rejects_negative_set_scores(value):
    with pytest.raises(ValueError, match="must be non-negative"):
        match_status(value, 0, 3)

@fast_fuzz
@given(best_of=st.integers(min_value=-20, max_value=20).filter(lambda n: n <= 0 or n % 2 == 0))
def test_match_status_requires_positive_odd_best_of(best_of):
    with pytest.raises(ValueError, match="best_of must be a positive odd integer"):
        match_status(0, 0, best_of)
```